

Informatika 3

Prednáška 5: Polymorfizmus

Michal Mrena



ŽILinskÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Kompozícia

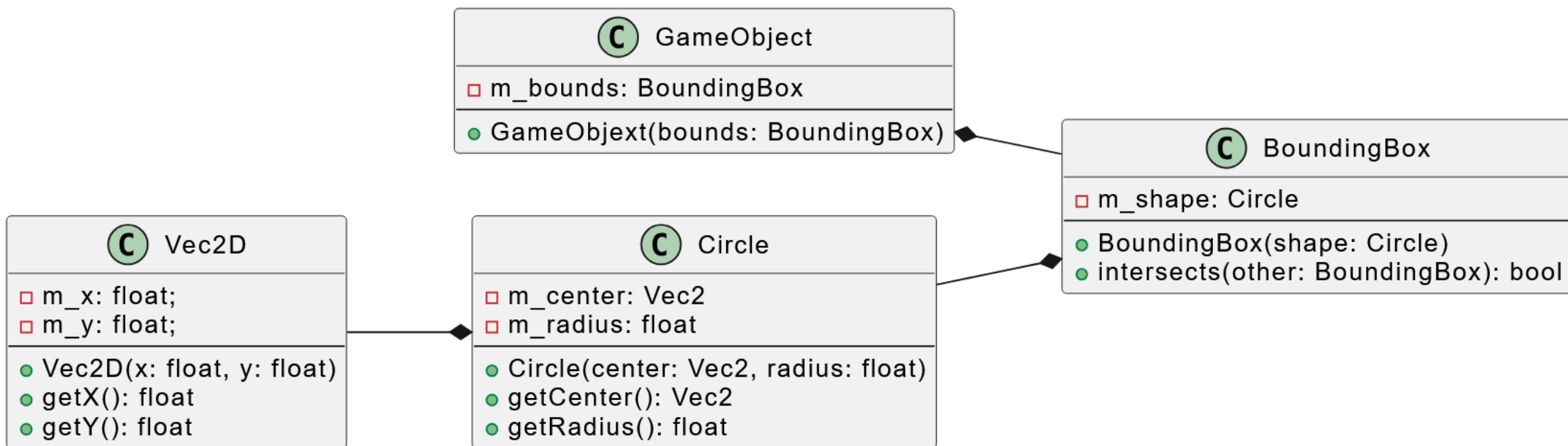




Barracks
Armor: 5
1200 / 1200

Kompozícia

- Každý objekt v hre má svoje ohraničenie – angl. [bounding box](#)
- Navrhujeme triedu reprezentujúci objekt v hre
- Pre jednoduchosť budeme pracovať v dvojrozmernom priestore



```

class Vec2 {
public:
    Vec2(float x, float y);
    float getX() const;
    float getY() const;
private:
    float m_x;
    float m_y;
};

Vec2::Vec2(float x, float y) :
    m_x(x),
    m_y(y) {

}

float Vec2::getX() const {
    return m_x;
}

float Vec2::getY() const {
    return m_y;
}

```

```

class Circle {
public:
    Circle(Vec2 center, float radius);
    Vec2 getCenter() const;
    float getRadius() const;
private:
    Vec2 m_center;
    float m_radius;
};

Circle::Circle(Vec2 center, float radius) :
    m_center(center),
    m_radius(radius) {

}

Vec2 Circle::getCenter() const {
    return m_center;
}

float Circle::getRadius() const {
    return m_radius;
}

```

```

class BoundingBox {
public:
    explicit BoundingBox(Circle shape);
    bool intersects(BoundingBox other) const;
private:
    Circle m_shape;
};

BoundingBox::BoundingBox(Circle shape) :
    m_shape(shape) {
}

bool BoundingBox::intersects(BoundingBox other) const {
    return m_shape.getRadius() + other.m_shape.getRadius()
        <= distance(
            m_shape.getCenter(),
            other.m_shape.getCenter());
}

```

```

class GameObject {
public:
    explicit GameObject(BoundingBox bounds);
private:
    BoundingBox m_bounds;
};

```

Pamäťová reprezentácia

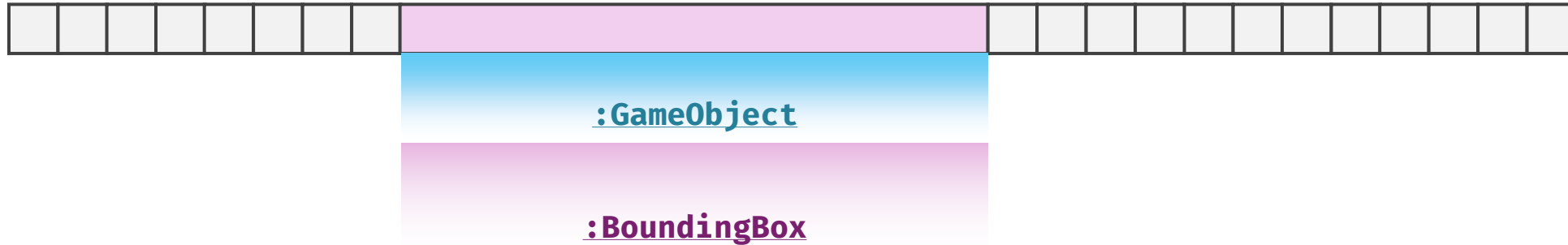
```
int main() {  
    std::cout << sizeof(GameObject) << "\n"; // 12  
    std::cout << alignof(GameObject) << "\n"; // 4  
}
```



1 bajt

Pamäťová reprezentácia

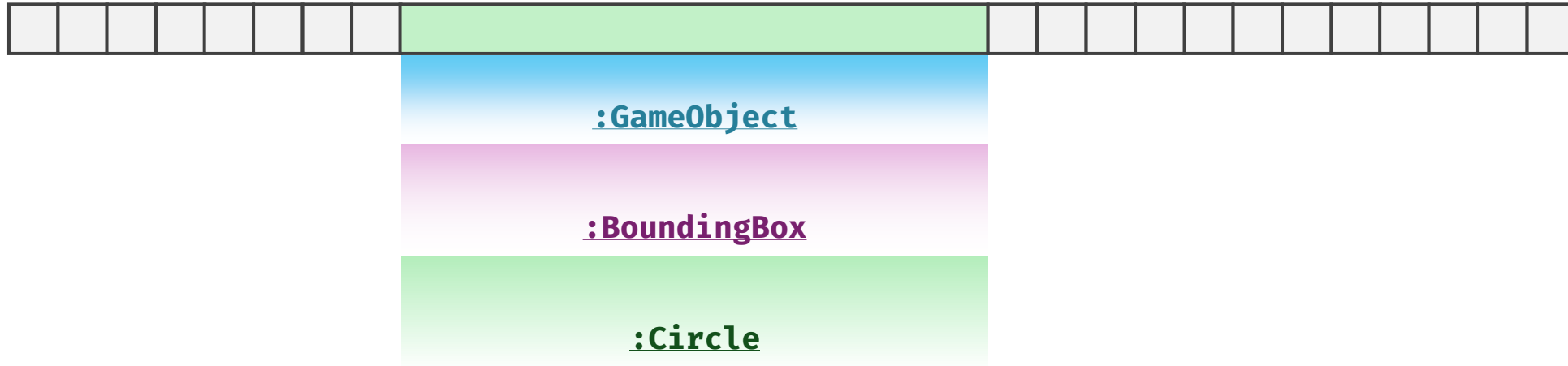
```
int main() {  
    std::cout << sizeof(GameObject) << "\n"; // 12  
    std::cout << alignof(GameObject) << "\n"; // 4  
}
```



1 bajt

Pamäťová reprezentácia

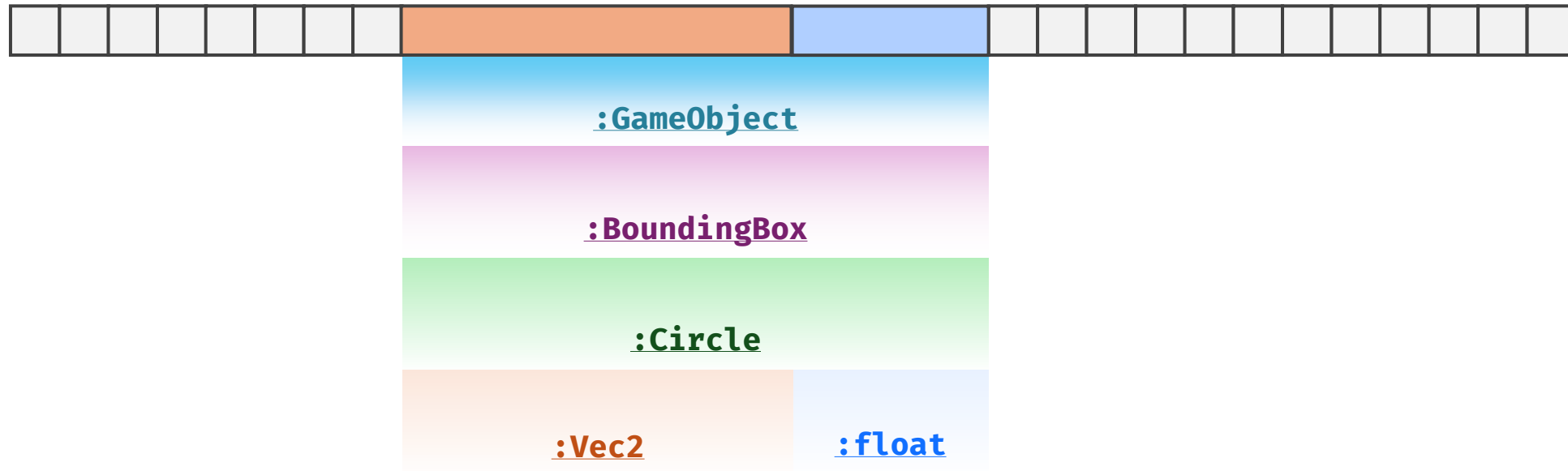
```
int main() {  
    std::cout << sizeof(GameObject) << "\n"; // 12  
    std::cout << alignof(GameObject) << "\n"; // 4  
}
```



1 bajt

Pamäťová reprezentácia

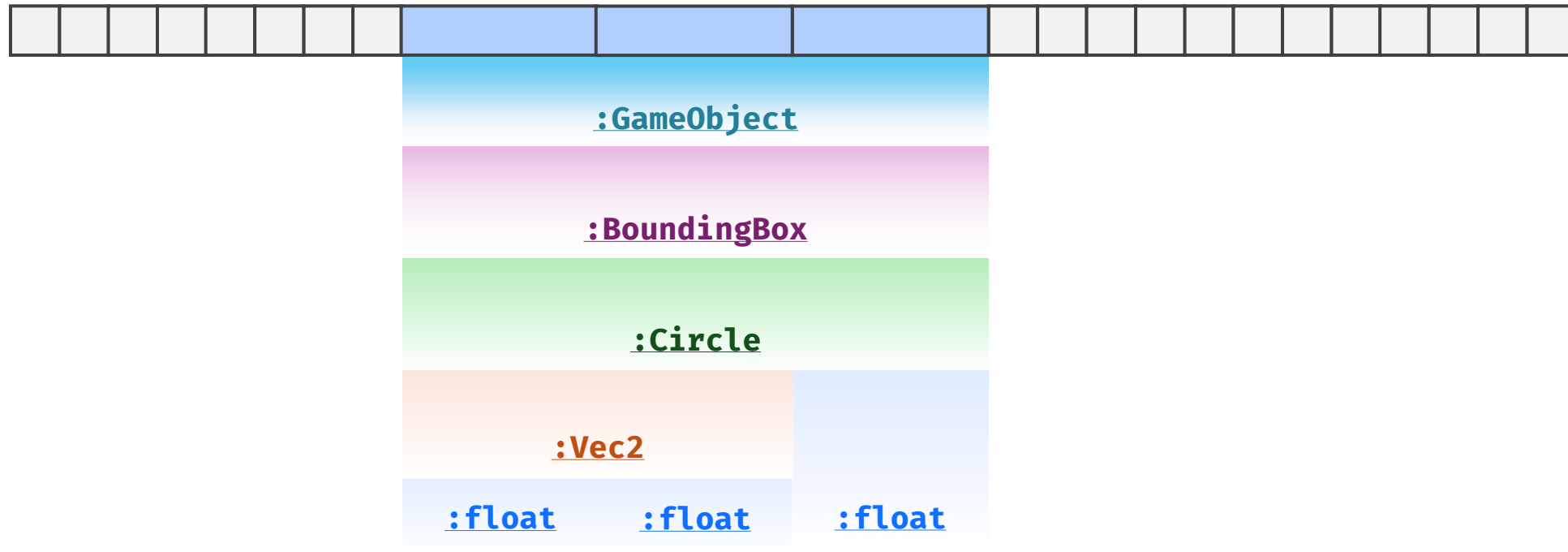
```
int main() {  
    std::cout << sizeof(GameObject) << "\n"; // 12  
    std::cout << alignof(GameObject) << "\n"; // 4  
}
```



1 bajt

Pamäťová reprezentácia

```
int main() {  
    std::cout << sizeof(GameObject) << "\n"; // 12  
    std::cout << alignof(GameObject) << "\n"; // 4  
}
```

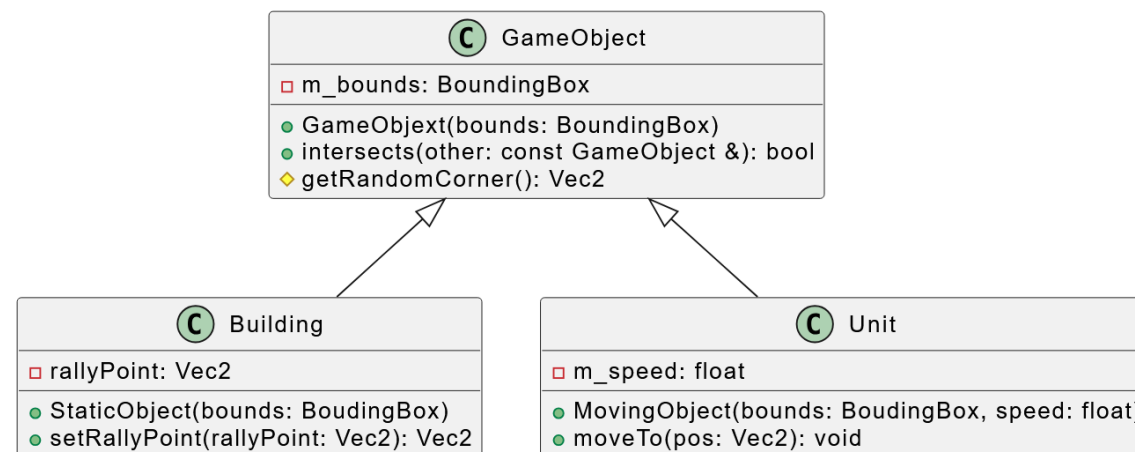


Jednoduchá dedičnosť



Jednoduchá dedičnosť

- V našom zjednodušení hry existujú dva typy herných objektov:
 - *Budovy* – vytvárajú jednotky
 - *Jednotky* – pohybujú sa po mape a plnia príkazy
- Pre oba typy objektov platí, že **sú** hernými objektami (*GameObject*)



Jednoduchá dedičnosť

```
class Building : public GameObject {
public:
    Building(BoundingBox bounds);
    void setRallyPoint(Vec2 rallyPoint);
private:
    Vec2 m_rallyPoint;
};

Building::Building(BoundingBox bounds) :
    GameObject(bounds),
    m_rallyPoint(this->getRandomCorner()) {
}

Vec2 Building::getRallyPoint() const {
    return m_rallyPoint;
}

void Building::setRallyPoint(Vec2 rallyPoint) {
    m_rallyPoint = rallyPoint;
}
```

```
class Unit : public GameObject {
public:
    Unit(BoundingBox bounds, float m_speed);
    void moveTo(Vec2 pos);
private:
    float m_speed;
};

Unit::Unit(BoundingBox bounds, float speed) :
    GameObject(bounds),
    m_speed(speed) {
}
```

Konštruktor predka môžeme zavolať syntaxou inicializácie členov. Ak to explicitne neurobíme, automaticky sa zavolá bezparametrický konštruktor. Ak takýto neexistuje, ide o chybu pri preklade.

Dedičnosť v jazyku C++

- Zoznam predkov triedy uvedený znakom : zapisujeme za názov triedy pred definíciu jej tela
- Pred názov predka uvádzame jeden z modifikátorov prístupu
- Pri vynechaní modifikátora sa použije predvolený pre
 - `class` je to `private` !
 - `struct` je to `public`

```
class Base {  
  
};  
  
class Child : public|protected|private Base {  
  
};
```

Modifikátory prístupu

- **public** je dedičnosť, ktorú poznáme z jazyka Java
 - Verejní členovia predka sú dostupní ako verejní členovia potomka
 - Chránení členovia sú dostupní v potomkovi a jeho potomkoch
- **protected**
 - **Verejní** a **chránení** členovia predka sú dostupní v potomkoch ako **chránení** členovia
 - Môže byť použitá pri implementácii [mixin tried](#)
 - Môže byť použitá pri implementácii návrhového vzoru [adapter](#)
- **private**
 - **Verejní** a **chránení** členovia predka sú dostupní v potomkovi ako **súkromní** členovia
 - Môže byť použitá na implementovanie vzťahu kompozície pri [metaprogramovaní](#)

Dedičnosť

- Medzi predkom a potomkom je tzv. IS-A vzťah – ak v kóde pracujeme s ukazovateľom alebo odkazom na predka, mali by sme byť schopný na tomto mieste použiť ľubovoľného z jeho potomkov – LSP

```
void logInfo(const GameObject &o);

int main() {
    Building b(mkBounds(Vec2(3,4), 10));
    Unit u(mkBounds(Vec2(3,4), 10), 2);
    logInfo(b);
    logInfo(u);
}
```

Pri použití `private` dedičnosti by bolo možné metódu `logInfo` zavolať iba z členských funkcií tried `Building` a `Unit`. Pri použití `protected` by to bolo možné aj v ich potomkoch.

Pamäťová reprezentácia



Pamäťová reprezentácia

```
class Building : public GameObject {
public:
    Building(BoundingBox bounds);
    void setRallyPoint(Vec2 rallyPoint);

private:
    Vec2 m_rallyPoint;
};
```

```
int main() {
    Building b(mkBounds(Vec2(3,4), 10));
    std::cout << sizeof(Building) << "\n"; // 20
    std::cout << alignof(Building) << "\n"; // 4
    std::cout << &b << '\n'; // 0x33a0fffb10
}
```

```
class GameObject {
public:
    GameObject(BoundingBox bounds);
    bool intersects(const GameObject &other) const;

private:
    BoundingBox m_bounds;
};
```

Ako bude vyzeráť pamäťová reprezentácia triedy `Building`?

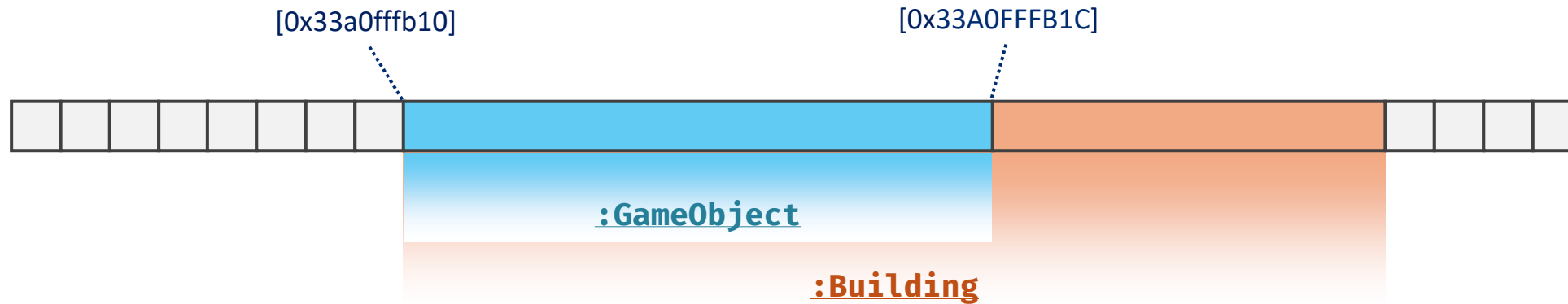
Pamäťová reprezentácia

- Inštancia triedy `Building` bude v sebe obsahovať *zdedených* členov (predka) triedy `GameObject` – sub-objekt



Pamäťová reprezentácia

- V pamäťovej reprezentácii je najprv uložený sub-objekt predka*
- Nasledovaný je členskými premennými potomka



Štandard však takéto rozloženie nevyžaduje! *

Pamäťová reprezentácia

- Každá inštancia *musí* zaberať aspoň jeden bajt (*prečo?*)
- Ak predok nemá žiadne členské premenné (teda vlastne nepotrebuje žiadne miesto) jeho sub-objekt nezaberie žiadne miesto – využíva sa v technike empty base optimization
- Využíva sa pri metaprogramovaní
- „The order in which the base class subobjects are allocated in the most derived object (6.7.2) is unspecified.“ [class.derived.general]

Konverzie – upcast

- Ukazovateľ/odkaz na potomka je **implicitne konvertovateľný** na ukazovateľ/odkaz na predka (tzv. upcast) *(prečo?)*
- Pre explicitnú konverziu (ktorá však nie je nutná) je možné použiť `static_cast`

```
void logInfo(const GameObject &o);

int main() {
    Building b(mkBounds(Vec2(3,4), 10));
    Unit u(mkBounds(Vec2(3,4), 10), 2);
    logInfo(b);
    logInfo(u);
}
```

```
void logInfo(const GameObject &o);

int main() {
    Building b(mkBounds(Vec2(3,4), 10));
    Unit u(mkBounds(Vec2(3,4), 10), 2);
    logInfo(static_cast<const GameObject&>(b));
    logInfo(static_cast<const GameObject&>(u));
}
```

Konverzie – downcast

- Pretypovanie z ukazovateľa/odkazu na predka na ukazovateľ/odkaz na potomka je možné iba explicitne
- V doposiaľ uvedených príkladoch je to možné len s použitím `static_cast`
- Toto pretypovanie však nerobí kontrolu skutočného typu za behu programu !
- Preto ho používame iba ak sme si istý, že pretypovanie je bezpečné (často pri generickom programovaní v spojení so šablónami)

Konverzie – downcast

```
void logInfo(const GameObject &o) {  
    const Building &b =  
        static_cast<const Building&>(o);  
  
    const Vec2 rp = b.getRallyPoint();  
    // ...  
}  
  
int main() {  
    Building b(mkBounds(Vec2(3,4), 10));  
    logInfo(b);  
}
```

```
void logInfo(const GameObject &o) {  
    const Building &b =  
        static_cast<const Building&>(o);  
  
    const Vec2 rp = b.getRallyPoint();  
    // ...  
}  
  
int main() {  
    Unit u(mkBounds(Vec2(3,4), 10), 2);  
    logInfo(u);  
}
```


Konverzie – downcast

```
void logInfo(const GameObject &o) {  
    const Building &b =  
        static_cast<const Building&>(o);  
  
    const Vec2 rp = b.getRallyPoint();  
    // ...  
}  
  
int main() {  
    Building b(mkBounds(Vec2(3,4), 10));  
    logInfo(b);  
}
```

Zlý kód, ale bude fungovať.

```
void logInfo(const GameObject &o) {  
    const Building &b =  
        static_cast<const Building&>(o);  
  
    const Vec2 rp = b.getRallyPoint();  
    // ...  
}  
  
int main() {  
    Unit u(mkBounds(Vec2(3,4), 10), 2);  
    logInfo(u);  
}
```

Zlý kód, a nebude ani fungovať.

Object slicing

- Inštanciu typu predka je možné inicializovať kopírovacím konštruktorom z hodnoty typu potomka
- Odkaz na potomka je implicitne konvertovateľný na odkaz na predka
- V príklade *strácame* časť objektu – prichádza k tzv. object slicing
- Za istých okolností môže viesť k nekonzistentnému stavu (pozri odkaz)

```
void serialize(GameObject o);

int main() {
    Building b(mkBounds(Vec2(3,4), 10));
    serialize(b);
}
```

Parameter o bude inicializovaný implicitne vygenerovaným kopírovacím konštruktorom:

```
GameObject(const GameObject &other);
```

Virtuálne členské funkcie



Virtuálne členské funkcie

- Pri práci s dedičnosťou chceme často prekryť (špecializovať) členské funkcie predka
- V jazyku Java sme mohli prekryť *ktorúkoľvek** verejnú alebo chránenú nestatickú metódu
- V jazyku C++ môžeme prekryť iba členské funkcie označené klúčovým slovom `virtual`
- Členské funkcie, ktoré prekrývajú funkcie predka označujeme klúčovým slovom `override` (od štandardu C++11, predtým sme aj prekrývajúce metódy označovali klúčovým slovom `virtual`)

Výnimkou je metóda označená klúčovým slovom `final`. *

Virtuálne členské funkcie

```
class GameObject {  
public:  
    GameObject(BoundingBox bounds);  
    virtual std::string getInfo() const;  
    bool intersects(const GameObject &other) const;  
  
protected:  
    Vec2 getRandomCorner() const;  
  
private:  
    BoundingBox m_bounds;  
};  
  
std::string GameObject::getInfo() const {  
    return "Generic game object";  
}
```

Virtuálne členské funkcie

```
class Building : public GameObject {
public:
    Building(BoundingBox bounds);
    std::string getInfo() const override;
    Vec2 getRallyPoint() const;
    void setRallyPoint(Vec2 rallyPoint);

private:
    Vec2 m_rallyPoint;
};

std::string Building::getInfo() const {
    return "Building object";
}
```

```
class Unit : public GameObject {
public:
    Unit(BoundingBox bounds, float m_speed);
    std::string getInfo() const override;
    void moveTo(Vec2 pos);

private:
    float m_speed;
};

std::string Unit::getInfo() const {
    return GameObject::getInfo() + ";Unit object";
}
```

Pri špecializácii funkcií predka potrebujeme často v špecializovanej funkcii zavolať implementáciu funkcie z predka. V takom prípade uvádzame pred názvom funkcie názov triedy – podobne ako pri volaní statickej členskej funkcie.

Virtuálne členské funkcie

```
class Unit : public GameObject {  
public:  
    Unit(BoundingBox bounds, float m_speed);  
    std::string getInfo() const override;  
    void moveTo(Vec2 pos);  
  
private:  
    float m_speed;  
};  
  
std::string Unit::getInfo() const {  
    return getInfo() + ";Unit object";  
}
```

Čo sa stane, ak neuvedieme predka?

Virtuálne členské funkcie

- Pri **nepriamom prístupe** a volaní virtuálnej funkcie sa vždy zavolá najšpecifickejšia implementácia funkcie podľa skutočného typu inštancie, ku ktorej nepriamo pristupujeme – o tom, ktorá funkcia sa tak rozhoduje až za behu programu (angl. **dynamic dispatch**)
- Pri **priamom prístupe** (pri práci s hodnotou) alebo volaní nie virtuálnej funkcie sa o tom, ktorá funkcia sa zavolá rozhoduje už pri preklade (angl. **static dispatch**)

```
void describeObject(const GameObject &o) {  
    std::cout << o.getInfo() << "\n";  
}  
int main() {  
    Unit u(mkBounds(Vec2(3,4), 10), 2);  
    Building b(mkBounds(Vec2(3,4), 10));  
    describeObject(u); // Generic game object;Unit object  
    describeObject(b); // Building object  
    std::cout << b.getInfo() << "\n";  
}
```

dynamic dispatch

static dispatch

Volanie členských funkcií

- **Priame volanie** – funkciu voláme priamo nad objektov
- **Nepriame volanie** – členskú funkciu voláme cez ukazovateľ alebo odkaz
- Pri priamom volaní virtuálnej členskej funkcie sa použije static dispatch napriek tomu, že je funkcia virtuálna – kompilátor totiž presne vie, ktorú implementáciu zvoliť

	Priame volanie	Nepriame volanie
Virtuálna funkcia	static dispatch	dynamic dispatch
Nie virtuálna funkcia	static dispatch	static dispatch

Čisté (abstraktné) virtuálne členské funkcie

- V niektorých situáciách vieme, že každá inštancia triedy bude mať nejakú funkciu, ale nie vždy vieme funkciu hneď implementovať
- Príklad: na každý objekt v hre môžeme kliknúť, čo sa stane po kliknutí závisí od typu objektu
- Deklarujeme preto čistú virtuálnu funkciu (angl. pure virtual)
- Syntax = `= 0` vyjadruje, že funkcia nebude mať v danej triede definíciu

```
class GameObject {  
public:  
    GameObject(BoundingBox bounds);  
    virtual void onClick() = 0;  
    virtual std::string getInfo() const;  
    bool intersects(const GameObject &other) const;  
  
protected:  
    Vec2 getRandomCorner() const;  
  
private:  
    BoundingBox m_bounds;  
};
```

Abstraktné triedy

- Trieda, ktorá ma aspoň jednu čistú virtuálnu členskú funkciu je **abstraktná trieda** – nie je možné vytvoriť z nej inštanciu
- Potomkovia abstraktnej triedy musia prekryť všetky čisté virtuálne funkcie alebo byť sami abstraktnými triedami

```
int main() {  
    GameObject o(mkBounds(Vec2(3,4), 4));  
}
```

```
main.cpp: In function 'int main()':  
main.cpp:227:16: error: cannot declare variable 'o' to be of abstract type 'GameObject'  
227 |     GameObject o(mkBounds(Vec2(3,4), 4));
```

Abstraktné triedy

- V triede `Unit` chceme ukladať sériu akcií, ktoré má jednotka vykonať
- Ak ak chceme akcie rozumne ukladať, potrebujú spoločného predka

```
class Action {  
public:  
    virtual bool execute() = 0;  
};
```

```
void Unit::addAction(Action &a) {  
    m_actions.push_back(&a);  
}
```

```
class Unit : public GameObject {  
public:  
    Unit(BoundingBox bounds, float m_speed);  
    void onClick() override;  
    std::string getInfo() const override;  
    void moveTo(Vec2 pos);  
    void addAction(Action &a);  
  
private:  
    std::vector<Action*> m_actions;  
    float m_speed;  
};
```

Abstraktné triedy

```
class ActionMove : public Action {
public:
    ActionMove(Unit &u, Vec2 pos);
    bool execute() override;

private:
    Unit *m_unit;
    Vec2 m_pos;
};

ActionMove::ActionMove(Unit &u, Vec2 pos) :
    m_unit(&u),
    m_pos(pos) {
}

bool ActionMove::execute() {
    m_unit->moveTo(m_pos);
    return true;
}
```

```
class ActionAttack : public Action {
public:
    ActionAttack(Unit &target);
    ~ActionAttack();
    bool execute() override;

private:
    Unit *m_target;
    AttackContext *m_context;
};

ActionAttack::ActionAttack(Unit &target) :
    m_target(&target),
    m_context(AttackContext::createDefault()) {
}

ActionAttack::~~ActionAttack() {
    delete m_context;
}

bool ActionAttack::execute() {
    m_target->doDamage(*m_context);
    return true;
}
```

Abstraktné triedy a rozhrania

- Jazyk C++ **nepozná** pojem rozhranie vo význame, ktorý poznáme z jazyka Java
- Za rozhranie môžeme považovať abstraktnú triedu, ktorej všetky členské funkcie sú čisté virtuálne a ktorá nemá žiadne členské premenné
- Používanie prefixu `I*` je – podobne ako v jazyku Java – vecou konvencie
- Abstraktnú triedu `Action` môžeme považovať za rozhranie

Virtuálny deštruktor

- Inštancie triedy `Unit` by pri svojom zániku mali zmazať všetky nevykonané akcie

```
class Action {  
public:  
    virtual bool execute() = 0;  
};
```

```
Unit::~~Unit() {  
    for (Action *a : m_actions) {  
        delete a;  
    }  
}
```

Ako vyzerá deštruktor triedy `Action`?
V čom je problém?

```
class ActionAttack : public Action {  
public:  
    ActionAttack(Unit &target);  
    ~ActionAttack();  
    bool execute() override;  
  
private:  
    Unit *m_target;  
    AttackContext *m_context;  
};  
  
ActionAttack::ActionAttack(Unit &target) :  
    m_target(&target),  
    m_context(new AttackContext()) {  
}  
  
ActionAttack::~~ActionAttack() {  
    delete m_context;  
}  
  
bool ActionAttack::execute() {  
    m_target->doDamage(*m_context);  
    return true;  
}
```

Virtuálny deštruktor

- Inštancie triedy `Unit` by pri svojom zániku mali zmazať všetky nevykonané akcie

```
Unit::~~Unit() {  
    for (Action *a : m_actions) {  
        delete a;  
    }  
}
```

Triedy `Action` Má automaticky vygenerovaný prázdny deštruktor.

Deštruktor triedy `Unit` zavolá pri mazaní akcií iba daný prázdny deštruktor. Prečo?

```
class ActionAttack : public Action {  
public:  
    ActionAttack(Unit &target);  
    ~ActionAttack();  
    bool execute() override;  
  
private:  
    Unit *m_target;  
    AttackContext *m_context;  
};  
  
ActionAttack::ActionAttack(Unit &target) :  
    m_target(&target),  
    m_context(AttackContext::createDefault()) {  
}  
  
ActionAttack::~~ActionAttack() {  
    delete m_context;  
}  
  
bool ActionAttack::execute() {  
    m_target->doDamage(*m_context);  
    return true;  
}
```


Virtuálny deštruktor

- Inštancie triedy `Unit` by pri svojom zániku mali zmazať všetky nevykonané akcie

```
Unit::~~Unit() {  
    for (Action *a : m_actions) {  
        delete a;  
    }  
}
```

Keďže deštruktory triedy `Action` a jej potomkov nie sú virtuálne, použije sa **static dispatch** a zavolá sa iba prázdny deštruktor triedy `Action`.

Výsledkom je únik pamäte v triede `ActionContext`.

```
class ActionAttack : public Action {  
public:  
    ActionAttack(Unit &target);  
    ~ActionAttack();  
    bool execute() override;  
  
private:  
    Unit *m_target;  
    AttackContext *m_context;  
};  
  
ActionAttack::ActionAttack(Unit &target) :  
    m_target(&target),  
    m_context(AttackContext::createDefault()) {  
}  
  
ActionAttack::~~ActionAttack() {  
    delete m_context;  
}  
  
bool ActionAttack::execute() {  
    m_target->doDamage(*m_context);  
    return true;  
}
```

Virtuálny deštruktor

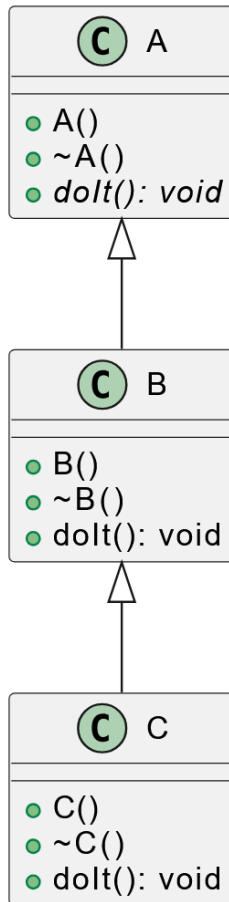
- Ak trieda definuje virtuálne funkcie je *vždy* potrebné definovať aj **virtuálny deštruktor** – aj keď predok, ktorý ho definuje často deštruktor nepotrebuje
- Virtuálny deštruktor vynúti **dynamic dispatch** pri jeho volaní – pri mazaní cez ukazovateľ na predka sa zavolajú aj deštruktory potomkov

```
class Action {  
public:  
    virtual ~Action() = default;  
    virtual bool execute() = 0;  
};
```

Syntax = `default` povie kompilátoru, aby vygeneroval prázdnu **definíciu** deštruktora.

Prázdny deštruktor by sme mohli definovať manuálne, ale je to zbytočné.

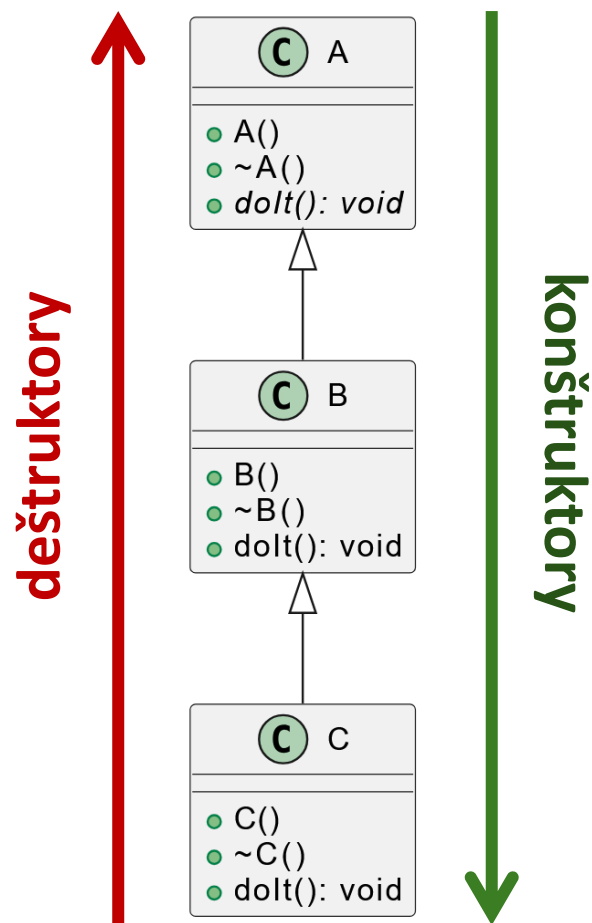
Konštruktor a deštruktor



```
int main() {
    C *c = new C();
    delete c;
}
```

V akom poradí sa zavolajú
konštruktory a v akom
deštruktory?

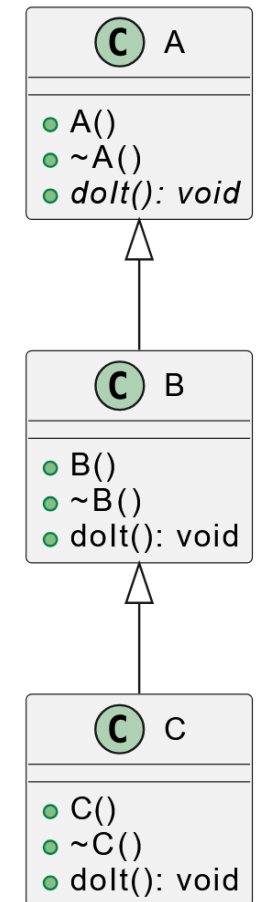
Konštruktor a deštruktor



```
int main() {  
    C *c = new C();  
    delete c;  
}
```

Konštruktor a deštruktor

- Pri vzniku inštancie sa najprv volajú konštruktory predkov až následne konštruktory potomkov
- Pri zániku inštancie sa najprv volajú deštruktory potomkov až následne deštruktory predkov
- Volania všetkých členských funkcií – **vrátane virtuálnych** – používa v konštruktore a deštruktore **static dispatch**
- Ak potrebujeme pri inicializácii správanie virtuálnych metód, môžeme použiť napríklad jednoduchú statickú **továrenskú metódu**



Dynamické pretypovanie

- Ak má trieda aspoň jednu virtuálnu metódu, môžeme pri pretypovaní použiť dynamické pretypovanie – `dynamic_cast`
- Na rozdiel od statického pretypovania robí kontrolu skutočného typu za behu

```
void logInfo(const GameObject &o) {  
    const Building &b =  
        dynamic_cast<const Building&>(o);  
    const Vec2 rp = b.getRallyPoint();  
}  
  
int main() {  
    Building b(mkBounds(Vec2(3,4), 10));  
    logInfo(b);  
}
```

```
void logInfo(const GameObject &o) {  
    const Building &b =  
        dynamic_cast<const Building&>(o);  
    const Vec2 rp = b.getRallyPoint();  
}  
  
int main() {  
    Unit u(mkBounds(Vec2(3,4), 10), 2);  
    logInfo(u);  
}
```

terminate called after throwing an instance
of 'std::bad_cast'
what(): std::bad_cast

Dynamické pretypovanie

- Pri dynamickom pretypovaní pracujeme s odkazom alebo ukazovateľom
- Ak pretypovanie zlyhá pri odkaze, dostaneme výnimku, ak zlyhá pri ukazovateli, dostaneme `nullptr`
- Pri pretypovaní je potrebné zachovať rovnaký `const` kvalifikátor

```
void logInfo(const GameObject &o) {  
    const Building *b =  
        dynamic_cast<const Building*>(&o);  
    const Vec2 rp = b->getRallyPoint();  
}  
int main() {  
    Building b(mkBounds(Vec2(3,4), 10));  
    logInfo(b);  
}
```

```
void logInfo(const GameObject &o) {  
    const Building *b =  
        dynamic_cast<const Building*>(&o);  
    if (b != nullptr) {  
        const Vec2 rp = b->getRallyPoint();  
    }  
}  
int main() {  
    Unit u(mkBounds(Vec2(3,4), 10), 2);  
    logInfo(u);  
}
```

Viacnásobná dedičnosť



Viacnásobná dedičnosť

- Zavedieme abstraktnú triedu `Spell`, ktorou budeme reprezentovať kúzla
- Inštancie tejto triedy sa vedia aplikovať na inštancie triedy `Unit`
- Následne budeme chcieť pridať triedu `StasisTrap` – objekt v hre, ak naň vojde jednotka, aplikuje sa na ňu kúzlo paralýzy

```
class Spell {  
public:  
    Spell(int cooldown);  
    virtual ~Spell();  
    virtual void apply(Unit &u) = 0;  
    int getCooldown() const;  
private:  
    int m_cooldown;  
};
```

- `StasisTrap` tak je herný objekt a zároveň je kúzlo

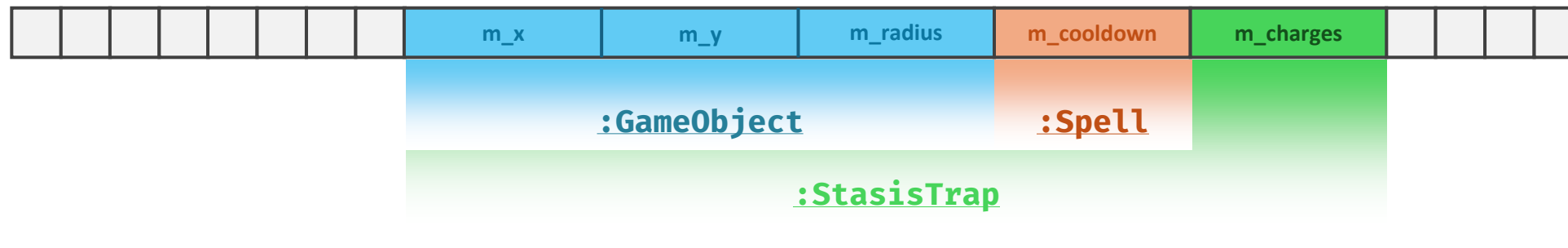


Viacnásobná dedičnosť

- Trieda môže mať v jazyku C++ ľubovoľný počet predkov
- Jednotlivých predkov oddeľujeme čiarkou
- Pre každého uvádzame modifikátor prístupu
- Konštruktory predkov voláme rovnako ako pri jednoduchšej dedičnosti – poradie je dané poradím deklarácie predkov
- Podobné poradie platí pri automatickom volaní deštruktorov

```
class StasisTrap :  
    public GameObject,  
    public Spell  
{  
public:  
    StasisTrap(BoundingBox bounds);  
    void apply(Unit &u) override;  
  
private:  
    int m_charges;  
};  
  
StasisTrap::StasisTrap(BoundingBox bounds) :  
    GameObject(bounds),  
    Spell(30),  
    m_charges(4) {  
}
```

Pamäťová reprezentácia

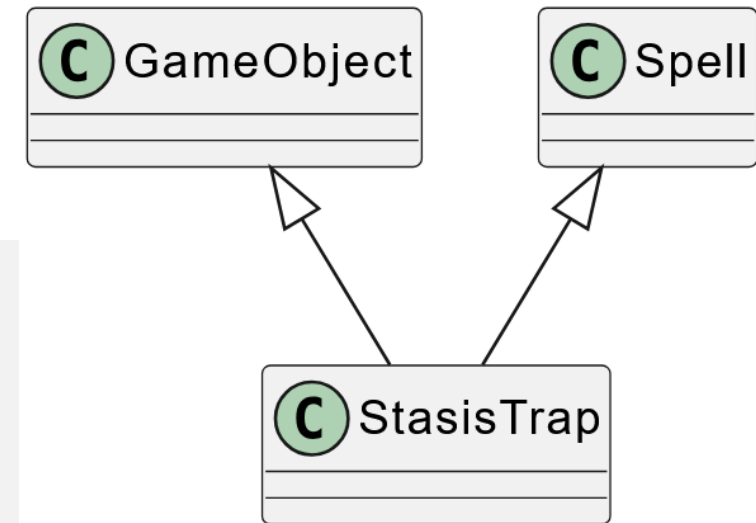


„The order of derivation is not significant except as specified by the semantics of initialization by constructor (`[class.base.init]`), cleanup (`[class.dtor]`), and storage layout (`[class.mem]`, `[class.access.spec]`).“

Dynamické pretypovanie

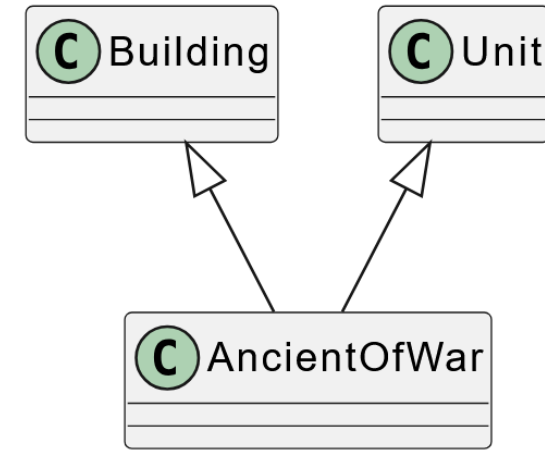
- Sidecast – pretypovanie na „rovnakej úrovni hierarchie dedičnosti“

```
void logInfo(Spell *s) {  
    GameObject *o = dynamic_cast<GameObject*>(s);  
    if (o != nullptr) {  
        std::cout << o->getInfo() << "\n";  
    }  
}  
  
int main() {  
    Spell s;  
    StasisTrap st;  
    logInfo(&s); //  
    logInfo(&st); // Generic game object  
}
```



Viacnásobná dedičnosť

- Po pridaní triedy `StasisTrap` chceme pridať triedu `AncientOfWar`
- Ide o herný objekt, ktorý sa dokáže správať aj ako budova aj ako jednotka
- Môžeme povedať, že `AncientOfWar` **je jednotka a je budova**
- Opäť môžeme využiť viacnásobnú dedičnosť
- V čom je táto situácia iná ako pri triede `StasisTrap`?

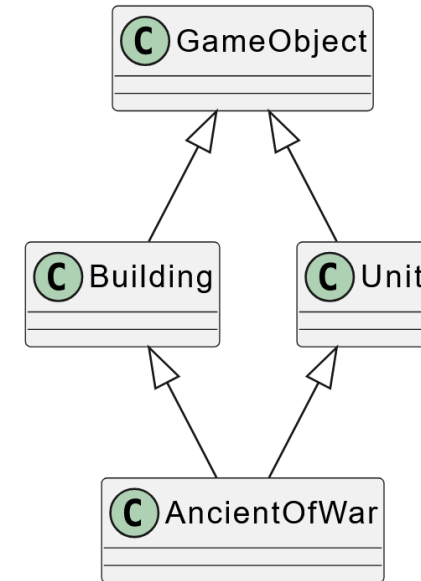


Viacnásobná dedičnosť

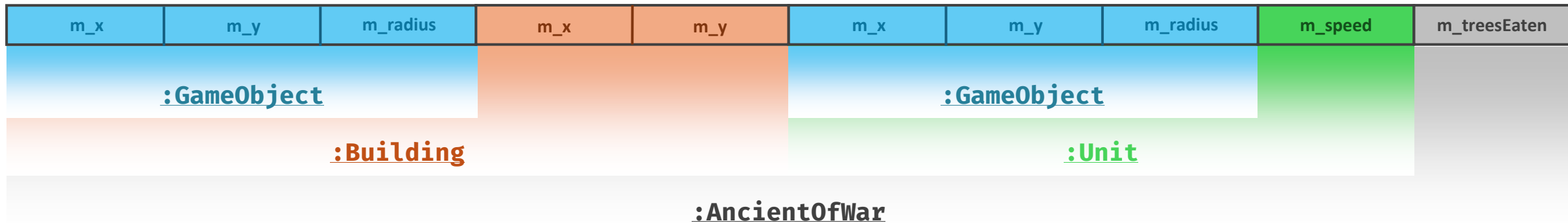
- Triedy **Building** a **Unit** sú **GameObject**
- Koľko sub-objektov typu **GameObject** bude v jednej inštancii typu **AncientOfWar**?



```
class GameObject {  
};  
  
class Building :  
    public GameObject {  
};  
  
class Unit :  
    public GameObject {  
};  
  
class AncientOfWar :  
    public Building,  
    public Unit  
{  
private:  
    int m_treesEaten;  
};
```



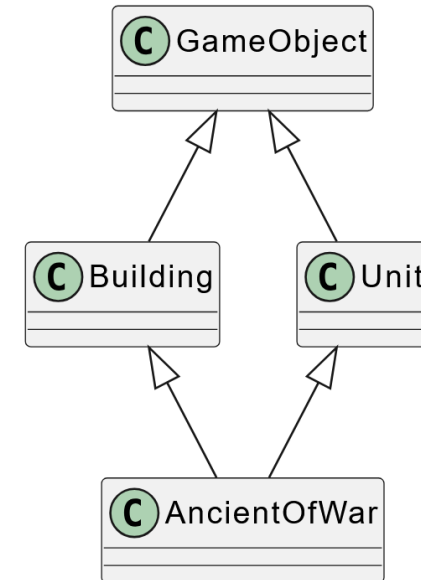
Viacnásobná dedičnosť



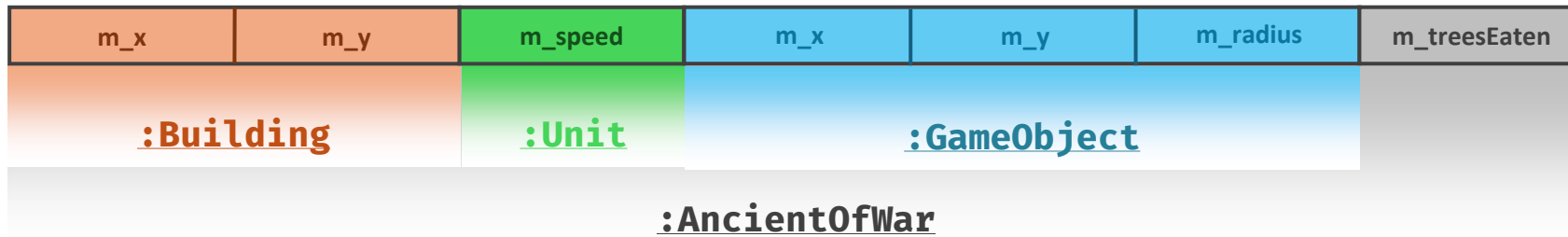
Virtuálna viacnásobná dedičnosť

- Sub-objekty budú dva, jeden súčasťou sub-objektu typu Building a druhý súčasťou typu Unit
- To sme ale, pravdepodobne, nechceli
- Riešením je použitie virtuálneho predka
- Kompilátor zabezpečí, že inštancia potomka bude obsahovať iba jeden sub-objekt virtuálneho predka

```
class GameObject {  
};  
  
class Bulding :  
    virtual public GameObject {  
};  
  
class Unit :  
    virtual public GameObject {  
};  
  
class AncientOfWar :  
    public Building,  
    public Unit {  
};
```



Viacnásobná dedičnosť



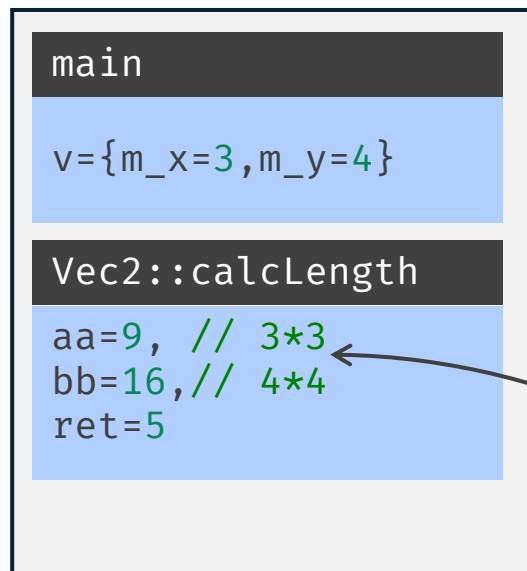
Prístup k členom



Prístup k členom

- Členské funkcie sú uložené v kódovom segmente procesu
- Členské premenné sú súčasťou inštancie uloženej na zásobníku alebo v halde
- Členská funkcia je v pamäti iba raz, inštancií môže byť niekoľko
- **Ako vie členská funkcia, s ktorou inštanciou má pracovať?**

```
class Vec2 {  
public:  
    Vec2(float x, float y);  
    float getX() const;  
    float getY() const;  
    float calcLength() const;  
  
public:  
    float m_x;  
    float m_y;  
};  
  
float Vec2::calcLength() const {  
    const float aa = m_x*m_x;  
    const float bb = m_y*m_y;  
    return std::sqrt(aa + bb);  
}  
  
int main() {  
    Vec2 v(3,4);  
    std::cout << v.calcLength() << "\n";  
}
```



Call stack

Ako sa sem dostanú tieto hodnoty?
Metóda (zdanlivo) nemá parameter?

```
class Vec2 {
public:
    Vec2(float x, float y);
    float getX() const;
    float getY() const;
    float calcLength() const;

public:
    float m_x;
    float m_y;
};

float Vec2::calcLength() const {
    const float aa = m_x*m_x;
    const float bb = m_y*m_y;
    return std::sqrt(aa + bb);
}

int main() {
    Vec2 v(3,4);
    std::cout << v.calcLength() << "\n";
}
```

Prístup k členom

- Každá členská funkcia má implicitný parameter – ukazovateľ na začiatok inštancie, nad ktorou sa metóda volá
- Parameter je v členskej funkcii dostupný kľúčovým slovom `this`



```
float Vec2::calcLength(const Vec2 *this) {  
    const float aa = t->m_x * t->m_x;  
    const float bb = t->m_y * t->m_y;  
    return std::sqrt(aa + bb);  
}
```

```
int main() {  
    Vec2 v(3,4);  
    std::cout << calcLength(&v) << "\n";  
}
```

Príklad je ilustračný. Uvedená ukážka by sa nepreložila. *

Prístup k členom

- Každá členská funkcia má implicitný parameter – ukazovateľ na začiatok inštalácie, nad ktorou sa metóda volá
- Parameter je v členskej funkcii dostupný kľúčovým slovom `this`
- (od C++23) Tento parameter môžeme uviesť explicitne špeciálnou syntaxou – [deducing this](#)



```
float Vec2::calcLength(this const Vec2 *t) {  
    const float aa = t->m_x * t->m_x;  
    const float bb = t->m_y * t->m_y;  
    return std::sqrt(aa + bb);  
}
```

```
int main() {  
    Vec2 v(3,4);  
    std::cout << v.calcLength() << "\n";  
}
```

`t == this == &v`

Virtuálne metódy

```
class GameObject {
public:
    virtual std::string getInfo() const;
};

std::string GameObject::getInfo() const {
    return "Game object";
}
```

```
class Building : public GameObject {
public:
    std::string getInfo() const override;
};

std::string Building::getInfo() const {
    return "Building object";
}
```

```
class Unit : public GameObject {
public:
    std::string getInfo() const override;
};

std::string Unit::getInfo() const {
    return "Unit object";
};
```

```
void printInfo(const GameObject &o) {
    std::cout << o.getInfo() << "\n";
}

int main() {
    Building b;
    std::cout << b.getInfo() << "\n";
}
```

Static dispatch

```
int main() {  
    Building b;  
    std::cout << b.getInfo() << "\n";  
}
```

Pracujeme priamo s hodnotou, kompilátor presne vie, ktorú funkciu zavolať.

this pointer

static dispatch

návratová hodnota (neskôr)

```
"main":  
    push    rbp  
    mov     rbp, rsp  
    push    rbx  
    sub     rsp, 56  
    mov     eax, OFFSET FLAT:"vtable for Building"+16  
    mov     QWORD PTR [rbp-56], rax  
    lea     rax, [rbp-48]  
    lea     rdx, [rbp-56]  
    mov     rsi, rdx  
    mov     rdi, rax  
    call    "Building::getInfo() const"  
    lea     rax, [rbp-48]  
    mov     rsi, rax  
    mov     edi, OFFSET FLAT:"std::cout"  
    call    "std::operator<<"
```

Príklad je špecifický pre System V x86-64 ABI používané kompilátormi GCC a Clang. *

Tabuľka virtuálnych metód – vtable

- Pre každú triedu, ktorá má aspoň jednu virtuálnu členskú funkciu sa generuje tabuľka virtuálnych funkcií
- Počas behu programu sa tabuľka použije na zavolanie správnej implementácie virtuálnej funkcie – dynamic dispatch
- Štandard nedefinuje, že sa má použiť tabuľka (aj keď práve takéto riešenie využívajú takmer všetky implementácie)
- Nasledujúce ukážky sú preto iba ilustratívne alebo špecifické pre System V ABI (x86-64) a Itanium C++ ABI

Tabuľka virtuálnych metód – vtable

```
class GameObject {
public:
    virtual ~GameObject() = default;
    virtual std::string getInfo() const;
private:
    __GameObjectVtable *__vptr;
};

class Building {
public:
    std::string getInfo() const override;
};

class Unit {
public:
    std::string getInfo() const override;
};
```

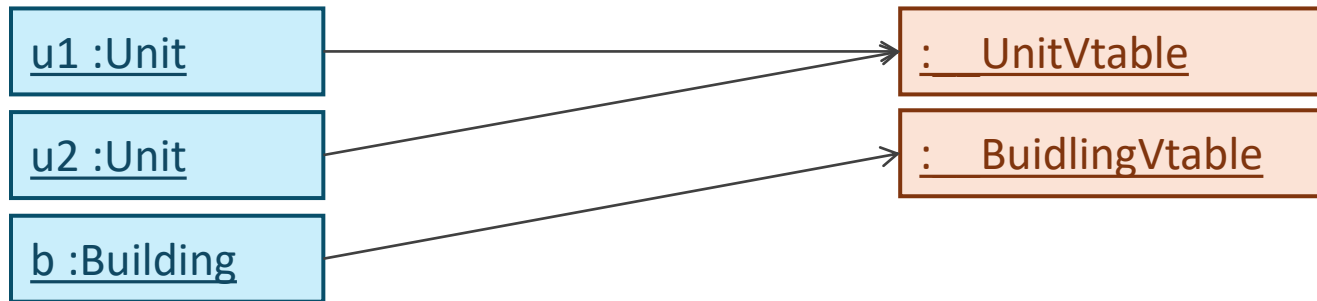
```
struct __GameObjectVtable {
    std::type_info *__t_info_ptr;
    void (*__deleting_destructor)(GameObject*);
    void (*__complete_destructor)(GameObject*);
    std::string (*getInfo)(const void*);
};

struct __BuildingVtable {
    std::type_info *__t_info_ptr;
    void (*__deleting_destructor)(Building*);
    void (*__complete_destructor)(Building*);
    std::string (*getInfo)(const void*);
};

struct __UnitVtable {
    std::type_info *__t_info_ptr;
    void (*__deleting_destructor)(Unit*);
    void (*__complete_destructor)(Unit*);
    std::string (*getInfo)(const void*);
};
```

Tabuľka virtuálnych metód – vtable

- Pre každú triedu, existuje jedna inštancia tabuľky virtuálnych funkcií
- Všetky inštancie danej triedy vlastnia ukazovateľ (ako prvú skrytú členskú premennú) na tabuľku danej triedy



Dynamic dispatch

```
void printInfo(const GameObject &o) {  
    std::cout << o.getInfo() << "\n";  
}  
  
int main() {  
    Building b;  
    std::cout << b.getInfo() << "\n";  
}
```

```
"printInfo(GameObject const&)":  
    push    rbp  
    mov     rbp, rsp  
    push    rbx  
    sub     rsp, 56  
    mov     QWORD PTR [rbp-56], rdi  
    mov     rax, QWORD PTR [rbp-56]  
    mov     rax, QWORD PTR [rax]  
    mov     rcx, QWORD PTR [rax]  
    lea     rax, [rbp-48]  
    mov     rdx, QWORD PTR [rbp-56]  
    mov     rsi, rdx  
    mov     rdi, rax  
    call    rcx
```

vtable pointer

this pointer

dynamic dispatch

⋮

`rcx` obsahuje adresu funkcie `Building::getInfo`.

návratová hodnota (neskôr)

Zhrnutie

- Dedičnosť s rôznymi modifikátormi prístupu a ich dôsledkami
- Virtuálne funkcie a ich správanie
- Virtuálny deštruktor
- Správanie virtuálnych funkcií v konštruktore a deštruktore

Ďakujem za pozornosť

Priestor na vaše otázky

